



# Special Course in Software Engineering '26

WSCI Framework

# Student Problem

*I changed my university password this morning. Now my Windows laptop won't connect to campus Wi-Fi, but my phone still works.*

# Give LLM everything

- Complete the code in the **bad\_wsci.py** file
- The point of the code is to take everything from all the files in the '**knowledge**' director and use them as its context.
- Then run the file and notice the model's output.
- The result might still be useful, but its context is large

## SELECT (manual way)

- Now, instead of giving the model everything under '**knowledge**' directory, give it only the specific relevant files.
- Update the code in **manual\_wsci.py** and then run it
- Note the model's output and the total characters of the context.
  - The context length would be much shorter
  - But the result might not be much different from the previous output. This shows that you do not need all those files to produce relevant result
  - Also, Qwen, or any modern models, are smart enough to know what is relevant.

# SELECT (slightly smarter)

- Earlier, you hardcoded the relevant files.
- A slightly smarter way would be to have Python handle this part
  - But that will still be a bit manual
- A truly smarter way would be to query a database and do semantic search and use the results to find what to select.
- Update the code in **smarter\_wsci.py** and run and note the output.
  - Very unlikely that the output would be much different from before.

# COMPRESS

- Create a function called **compress\_context()** that takes the context from the previous step and user's question as parameters.
- Use Qwen to compress that context, so that only the relevant information is extracted.
- Pass that result as the compressed context to the next Qwen call, where the student's problem is addressed.
- Note the output of that call. Again, it won't be much different from the earlier results.
  - This should again prove that despite the compressed context, you can get useful result

# WRITE

- This is nothing but saving the relevant information to some **file/artifact**
- Write the model's output to a file called **state.json**
  - Make sure the output is structured
- Update the whole program so that the model is always using the relevant information from **state.json** as part of its context

# ISOLATE

- **ISOLATE** can be as simple as creating different states/artifacts like below:

```
diagnostic_context = {  
    "problem": question,  
    "device": "Windows laptop",  
    "wifi_status": "operational"  
}
```

```
report_context = {  
    "total_wifi_cases": 37,  
    "resolved_cases": 29,  
    "unresolved_cases": 8  
}
```

- Then depending on the agent's task, the appropriate state is used
  - Here, also, Qwen can be used to classify the problem and then give that information back to your program to perform isolation

# That's all